

Universidad Privada Domingo Savio

Tecnología Web II

Entrega 10A

Configuración de Evaluación: CRUD y Ponderaciones

FASE 4: Horarios, Asistencias y Configuración de Evaluación

Abril 2026

Objetivos de Aprendizaje

- Crear un schema Zod que valide la suma de pesos = 100% a nivel de aplicación (no en PostgreSQL).
- Implementar API Routes CRUD con validación de integridad de ponderaciones por materia.
- Construir un componente de tabla editable con indicador visual en tiempo real de la suma total.
- Aplicar el patrón de permisos mixtos: admin tiene acceso total, docente solo a sus materias.
- Crear una vista de consulta de ponderaciones para estudiantes (solo lectura).

Prerrequisitos

- ✓ Entrega 8A completada: tabla configuracion_evaluacion creada con RLS habilitado.
- ✓ Entregas 9A-9B completadas: módulo de asistencias funcional.
- ✓ Al menos 2 materias con docente asignado en la base de datos.
- ✓ Componentes de shadcn/ui instalados: Dialog, Table, Select, Badge, Progress.

Teoría: Validación de Suma a Nivel de Aplicación

En la Entrega 8A decidimos que la validación de suma de pesos = 100% se haría a nivel de aplicación, no en PostgreSQL. Ahora veamos por qué y cómo implementarlo.

El Problema de la Validación de Suma en PostgreSQL

Pseudo-código: Por qué un CHECK no funciona

```
-- ESTO NO FUNCIONA en PostgreSQL:
CHECK (
  (SELECT SUM(peso) FROM configuracion_evaluacion
   WHERE materia_id = NEW.materia_id) = 100
)
```

Razones:

1. CHECK constraints NO pueden contener subqueries
2. Un CHECK opera sobre UNA FILA, no sobre un conjunto
3. Se evaluaría ANTES del INSERT, no después

Alternativa en PostgreSQL: TRIGGER AFTER INSERT/UPDATE

Pero esto tiene problemas:

- No puedes insertar la primera fila (parcial_1: 25%) porque la suma sería 25, no 100
- Tendrías que insertar TODAS las filas en una transacción y validar al final
- Los mensajes de error son poco amigables

Solución elegida: Validar en la API Route

- Permite guardar configuraciones parciales (suma < 100)
- Muestra advertencia visual pero no bloquea el guardado
- Bloquea solo al intentar 'finalizar' la configuración

Estrategia de Validación en 2 Niveles

Nivel	Que valida	Cuando se aplica	Mensaje de error
PostgreSQL CHECK	peso BETWEEN 0 AND 100	En cada INSERT/UPDATE individual	Error: peso fuera de rango
PostgreSQL UNIQUE	materia_id + tipo_evaluacion	En cada INSERT/UPDATE individual	Error: tipo duplicado

Nivel	Que valida	Cuando se aplica	Mensaje de error
API Route (parcial)	Cada campo individual (Zod)	Al crear/editar un registro	Error: campo invalido
API Route (total)	SUM(pesos) = 100 por materia	Al marcar como 'finalizada'	Advertencia: suma != 100

NOTA: Guardado parcial vs finalizado

Permitimos que el admin/docente agregue ponderaciones una por una sin exigir que la suma sea 100 inmediatamente. El indicador visual muestra la suma en tiempo real (verde si es 100, rojo si no). Solo mostramos un warning al guardar si la suma no es 100, pero NO bloqueamos el guardado.

Paso 1: Schema de Validación Zod

Crea el schema de validación para configuración de evaluación. Incluye la validación individual de cada registro y una función auxiliar para verificar la suma.

TypeScript - lib/validations/configuracion-evaluacion.ts (crear archivo nuevo)

```
import { z } from "zod";

// Schema para un registro individual
export const configEvaluacionSchema = z.object({
  materia_id: z
    .string()
    .uuid("Debe seleccionar una materia valida"),

  tipo_evaluacion: z.enum(
    [
      "parcial_1", "parcial_2", "parcial_3",
      "final", "practica", "proyecto", "extra",
    ],
    {
      errorMap: () => ({ message: "Tipo de evaluacion invalido" })
    }
  ),

  peso: z
    .number({ invalid_type_error: "El peso debe ser un numero" })
    .min(0, "El peso no puede ser negativo")
    .max(100, "El peso no puede superar 100")
    .multipleOf(0.01, "Maximo 2 decimales"),

  descripcion: z
    .string()
    .max(200, "Maximo 200 caracteres")
    .optional()
    .or(z.literal("")),

  obligatoria: z.boolean().default(true),
});

export type ConfigEvaluacionFormData = z.infer<
  typeof configEvaluacionSchema
>;

// — Tipos de evaluacion con labels —
export const TIPOS_EVALUACION = [
  { value: "parcial_1", label: "Parcial 1" },
  { value: "parcial_2", label: "Parcial 2" },
  { value: "parcial_3", label: "Parcial 3" },
  { value: "final", label: "Examen Final" },
];
```

```

    { value: "practica", label: "Practica / Laboratorio" },
    { value: "proyecto", label: "Proyecto Final" },
    { value: "extra", label: "Puntos Extra" },
  ];

  // — Funcion auxiliar: verificar suma —
  export function verificarSumaPesos(
    configs: { peso: number; tipo_evaluacion: string }[]
  ): {
    suma: number;
    esValida: boolean;
    mensaje: string;
  } {
    // Excluir "extra" de la suma obligatoria
    const configsObligatorias = configs.filter(
      (c) => c.tipo_evaluacion !== "extra"
    );
    const suma = configsObligatorias.reduce(
      (acc, c) => acc + c.peso, 0
    );
    const sumaRedondeada = Math.round(suma * 100) / 100;

    if (sumaRedondeada === 100) {
      return {
        suma: sumaRedondeada,
        esValida: true,
        mensaje: "Configuración completa (100%)",
      };
    }

    if (sumaRedondeada < 100) {
      return {
        suma: sumaRedondeada,
        esValida: false,
        mensaje: `Faltan ${((100 - sumaRedondeada).toFixed(2))}% por asignar`,
      };
    }

    return {
      suma: sumaRedondeada,
      esValida: false,
      mensaje: `Excede por ${((sumaRedondeada - 100).toFixed(2))}%`,
    };
  }

```

TIP: Excluir 'extra' de la suma

El tipo 'extra' representa puntos adicionales/bonus que no son parte de la nota regular. Por eso se excluye del cálculo de suma = 100%. Un estudiante podría tener 100% de nota base + 5% de puntos extra = 105% total.

ADVERTENCIA: `z.number().multipleOf(0.01)`

La validación `multipleOf(0.01)` asegura que el peso tenga máximo 2 decimales, coincidiendo con `NUMERIC(5,2)` de PostgreSQL. Sin esta validación, un peso como 33.333 pasaría Zod pero fallaría en la BD.

Paso 2: API Route Admin - GET y POST

El GET lista todas las configuraciones de evaluación con JOIN a materias. El POST crea un nuevo registro validando con Zod y advirtiéndolo si la suma se desequilibra.

```
TypeScript - app/api/admin/evaluacion/route.ts (crear archivo nuevo)
import { createClient } from "@lib/supabase/server";
import { NextResponse } from "next/server";
import {
  configEvaluacionSchema,
  verificarSumaPesos,
} from "@lib/validations/configuracion-evaluacion";

// — GET: Listar configuraciones (opcionalmente por materia) —
export async function GET(request: Request) {
  const supabase = await createClient();

  const { data: { user }, error: authError } = await supabase.auth.getUser();
  if (authError || !user) {
    return NextResponse.json({ error: "No autenticado" }, { status: 401 });
  }

  const { data: perfil } = await supabase
    .from("usuarios_perfil")
    .select("rol")
    .eq("id", user.id)
    .single();

  if (perfil?.rol !== "admin") {
    return NextResponse.json({ error: "Acceso denegado" }, { status: 403 });
  }

  // Opcional: filtrar por materia
  const { searchParams } = new URL(request.url);
  const materia_id = searchParams.get("materia_id");

  let query = supabase
    .from("configuracion_evaluacion")
    .select(`
      *,
      materias (id, nombre, codigo)
    `)
    .order("materia_id")
    .order("tipo_evaluacion");

  if (materia_id) {
    query = query.eq("materia_id", materia_id);
  }
}
```

```

}

const { data: configs, error } = await query;

if (error) {
  return NextResponse.json({ error: error.message }, { status: 500 });
}

// Calcular suma por materia
const materiasMap = new Map<string, any[]>();
(configs || []).forEach((cfg) => {
  const arr = materiasMap.get(cfg.materia_id) || [];
  arr.push(cfg);
  materiasMap.set(cfg.materia_id, arr);
});

const resumenPorMateria: Record<string, any> = {};
materiasMap.forEach((cfgs, mId) => {
  resumenPorMateria[mId] = verificarSumaPesos(cfgs);
});

return NextResponse.json({
  configuraciones: configs,
  resumenPorMateria,
});
}

// — POST: Crear configuración —
export async function POST(request: Request) {
  const supabase = await createClient();

  const { data: { user }, error: authError } = await supabase.auth.getUser();
  if (authError || !user) {
    return NextResponse.json({ error: "No autenticado" }, { status: 401 });
  }

  const { data: perfil } = await supabase
    .from("usuarios_perfil")
    .select("rol")
    .eq("id", user.id)
    .single();

  if (perfil?.rol !== "admin") {
    return NextResponse.json({ error: "Acceso denegado" }, { status: 403 });
  }

  const rawBody = await request.json();
  // Convertir peso a number si viene como string
  const bodyParsed = {

```

```

    ...rawBody,
    peso: typeof rawBody.peso === "string"
      ? parseFloat(rawBody.peso)
      : rawBody.peso,
  };

  const resultado = configEvaluacionSchema.safeParse(bodyParsed);
  if (!resultado.success) {
    return NextResponse.json(
      { error: resultado.error.errors[0].message },
      { status: 400 }
    );
  }

  const { data: nuevo, error } = await supabase
    .from("configuracion_evaluacion")
    .insert({
      materia_id: resultado.data.materia_id,
      tipo_evaluacion: resultado.data.tipo_evaluacion,
      peso: resultado.data.peso,
      descripcion: resultado.data.descripcion || null,
      obligatoria: resultado.data.obligatoria,
    })
    .select()
    .single();

  if (error) {
    if (error.code === "23505") {
      return NextResponse.json(
        { error: "Ya existe este tipo de evaluacion para esta materia" },
        { status: 409 }
      );
    }
    return NextResponse.json({ error: error.message }, { status: 500 });
  }

  // Verificar suma despues del insert
  const { data: todasConfigs } = await supabase
    .from("configuracion_evaluacion")
    .select("peso, tipo_evaluacion")
    .eq("materia_id", resultado.data.materia_id);

  const sumCheck = verificarSumaPesos(todasConfigs || []);

  return NextResponse.json(
    {
      configuracion: nuevo,
      sumaPesos: sumCheck,
    },
    { status: 201 }
  );

```

```
}
```

NOTA: Respuesta con verificación de suma

El POST no solo devuelve el registro creado, sino también el estado de la suma de pesos para esa materia. Esto permite al frontend actualizar el indicador visual sin hacer una segunda consulta.

Paso 3: API Route Admin - PUT y DELETE

```
TypeScript - app/api/admin/evaluacion/[id]/route.ts (crear archivo nuevo)
import { createClient } from "@lib/supabase/server";
import { NextResponse } from "next/server";
import {
  configEvaluacionSchema,
  verificarSumaPesos,
} from "@lib/validations/configuracion-evaluacion";

// — PUT: Editar configuración —
export async function PUT(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  const supabase = await createClient();

  const { data: { user }, error: authError } = await supabase.auth.getUser();
  if (authError || !user) {
    return NextResponse.json({ error: "No autenticado" }, { status: 401 });
  }

  const { data: perfil } = await supabase
    .from("usuarios_perfil").select("rol")
    .eq("id", user.id).single();

  if (perfil?.rol !== "admin") {
    return NextResponse.json({ error: "Acceso denegado" }, { status: 403 });
  }

  const rawBody = await request.json();
  const bodyParsed = {
    ...rawBody,
    peso: typeof rawBody.peso === "string"
      ? parseFloat(rawBody.peso) : rawBody.peso,
  };

  const resultado = configEvaluacionSchema.safeParse(bodyParsed);
  if (!resultado.success) {
    return NextResponse.json(
      { error: resultado.error.errors[0].message },
      { status: 400 }
    );
  }

  const { data: actualizado, error } = await supabase
    .from("configuracion_evaluacion")
```

```

    .update({
      tipo_evaluacion: resultado.data.tipo_evaluacion,
      peso: resultado.data.peso,
      descripcion: resultado.data.descripcion || null,
      obligatoria: resultado.data.obligatoria,
    })
    .eq("id", id)
    .select()
    .single();

    if (error) {
      if (error.code === "23505") {
        return NextResponse.json(
          { error: "Ya existe este tipo de evaluacion para esta materia" },
          { status: 409 }
        );
      }
      return NextResponse.json({ error: error.message }, { status: 500 });
    }

    // Verificar suma despues del update
    const { data: todasConfigs } = await supabase
      .from("configuracion_evaluacion")
      .select("peso, tipo_evaluacion")
      .eq("materia_id", resultado.data.materia_id);

    const sumCheck = verificarSumaPesos(todasConfigs || []);

    return NextResponse.json({ configuracion: actualizado, sumaPesos: sumCheck });
  }

  // — DELETE: Eliminar configuracion —
  export async function DELETE(
    _request: Request,
    { params }: { params: Promise<{ id: string }> }
  ) {
    const { id } = await params;
    const supabase = await createClient();

    const { data: { user }, error: authError } = await supabase.auth.getUser();
    if (authError || !user) {
      return NextResponse.json({ error: "No autenticado" }, { status: 401 });
    }

    const { data: perfil } = await supabase
      .from("usuarios_perfil").select("rol")
      .eq("id", user.id).single();

    if (perfil?.rol !== "admin") {
      return NextResponse.json({ error: "Acceso denegado" }, { status: 403 });
    }
  }

```

```

}

// Obtener materia_id antes de eliminar (para recalcular suma)
const { data: config } = await supabase
  .from("configuracion_evaluacion")
  .select("materia_id")
  .eq("id", id)
  .single();

const { error } = await supabase
  .from("configuracion_evaluacion")
  .delete()
  .eq("id", id);

if (error) {
  return NextResponse.json({ error: error.message }, { status: 500 });
}

// Recalcular suma
let sumCheck = null;
if (config?.materia_id) {
  const { data: restantes } = await supabase
    .from("configuracion_evaluacion")
    .select("peso, tipo_evaluacion")
    .eq("materia_id", config.materia_id);
  sumCheck = verificarSumaPesos(restantes || []);
}

return NextResponse.json({ mensaje: "Eliminado", sumaPesos: sumCheck });
}

```

TIP: Recalcular suma en DELETE

Al eliminar una configuración, la suma cambia. El DELETE obtiene el `materia_id` ANTES de borrar, luego recalcula y devuelve la nueva suma. Esto evita que el frontend muestre una suma desactualizada.

Paso 4: API Routes Docente y Estudiante

El docente puede ver y editar las ponderaciones de SUS materias. El estudiante solo puede ver las ponderaciones de las materias donde está inscrito.

TypeScript - app/api/docente/evaluacion/route.ts (crear archivo nuevo)

```
import { createClient } from "@lib/supabase/server";
import { NextRequest, NextResponse } from "next/server";
import { verificarSumaPesos } from "@lib/validations/configuracion-evaluacion";

export async function GET(request: NextRequest) {
  const supabase = await createClient();

  const { data: { user }, error: authError } = await supabase.auth.getUser();
  if (authError || !user) {
    return NextResponse.json({ error: "No autenticado" }, { status: 401 });
  }

  // Obtener materias del docente
  const { data: misMaterias } = await supabase
    .from("materias")
    .select("id, nombre, codigo")
    .eq("docente_id", user.id);

  if (!misMaterias || misMaterias.length === 0) {
    return NextResponse.json({ materias: [] });
  }

  const materiaIds = misMaterias.map((m) => m.id);

  const { data: configs } = await supabase
    .from("configuracion_evaluacion")
    .select("*")
    .in("materia_id", materiaIds)
    .order("tipo_evaluacion");

  // Agrupar por materia con suma
  const materiasConConfig = misMaterias.map((mat) => {
    const cfigs = (configs || []).filter(
      (c) => c.materia_id === mat.id
    );
    return {
      materia: mat,
      configuraciones: cfigs,
      sumaPesos: verificarSumaPesos(cfigs),
    };
  });
}
```

```

    return NextResponse.json({ materias: materiasConConfig });
}

```

TypeScript - app/api/estudiante/evaluacion/route.ts (crear archivo nuevo)

```

import { createClient } from "@lib/supabase/server";
import { NextResponse } from "next/server";
import { verificarSumaPesos } from "@lib/validations/configuracion-evaluacion";

export async function GET() {
  const supabase = await createClient();

  const { data: { user }, error: authError } = await supabase.auth.getUser();
  if (authError || !user) {
    return NextResponse.json({ error: "No autenticado" }, { status: 401 });
  }

  // Materias donde esta inscrito
  const { data: inscripciones } = await supabase
    .from("inscripciones")
    .select("materia_id, materias (id, nombre, codigo)")
    .eq("estudiante_id", user.id)
    .eq("estado", "inscrito");

  if (!inscripciones || inscripciones.length === 0) {
    return NextResponse.json({ materias: [] });
  }

  const materiaIds = inscripciones.map((i) => i.materia_id);

  const { data: configs } = await supabase
    .from("configuracion_evaluacion")
    .select("*")
    .in("materia_id", materiaIds)
    .order("tipo_evaluacion");

  const materias = inscripciones.map((insc) => {
    const cfigs = (configs || []).filter(
      (c) => c.materia_id === insc.materia_id
    );
    return {
      materia: insc.materias,
      configuraciones: cfigs,
      sumaPesos: verificarSumaPesos(cfigs),
    };
  });

  return NextResponse.json({ materias });
}

```

```
}
```

NOTA: Misma función verificarSumaPesos en 3 APIs

La función verificarSumaPesos se reutiliza en las 3 API Routes (admin, docente, estudiante). Esto garantiza que el cálculo de la suma sea consistente en todo el sistema. Si cambiamos la regla (por ejemplo, incluir 'extra'), solo editamos UN archivo.

Paso 5: Componente ConfigEvaluacionTable (Admin)

Este componente es el más complejo de la entrega. Combina una tabla CRUD con un indicador visual de la suma de pesos en tiempo real. El admin selecciona una materia y ve/edita sus ponderaciones.

TypeScript - components/dashboard/admin/config-evaluacion-table.tsx (Parte 1: Imports y Estado)

```
"use client";

import { useEffect, useState } from "react";
import { toast } from "sonner";
import { Plus, Pencil, Trash2, BarChart3, Check, X } from "lucide-react";

import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Label } from "@components/ui/label";
import { Switch } from "@components/ui/switch";
import {
  Table, TableBody, TableCell, TableHead,
  TableHeader, TableRow,
} from "@components/ui/table";
import {
  Dialog, DialogContent, DialogDescription,
  DialogHeader, DialogTitle, DialogTrigger,
} from "@components/ui/dialog";
import {
  Select, SelectContent, SelectItem,
  SelectTrigger, SelectValue,
} from "@components/ui/select";
import {
  AlertDialog, AlertDialogAction, AlertDialogCancel,
  AlertDialogContent, AlertDialogDescription,
  AlertDialogFooter, AlertDialogHeader, AlertDialogTitle,
  AlertDialogTrigger,
} from "@components/ui/alert-dialog";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Badge } from "@components/ui/badge";
import { Progress } from "@components/ui/progress";

import {
  configEvaluacionSchema,
  TIPOS_EVALUACION,
  verificarSumaPesos,
} from "@lib/validations/configuracion-evaluacion";

interface Materia { id: string; nombre: string; codigo: string; }
interface Config {
```

```

id: string;
materia_id: string;
tipo_evaluacion: string;
peso: number;
descripcion: string | null;
obligatoria: boolean;
materias: Materia;
}

```

TypeScript - config-evaluacion-table.tsx (Parte 2: Estado y Carga)

```

export default function ConfigEvaluacionTable() {
  const [configs, setConfigs] = useState<Config[]>([]);
  const [materias, setMaterias] = useState<Materia[]>([]);
  const [materiaFiltro, setMateriaFiltro] = useState("");
  const [cargando, setCargando] = useState(true);
  const [dialogAbierto, setDialogAbierto] = useState(false);
  const [guardando, setGuardando] = useState(false);
  const [editando, setEditando] = useState<Config | null>(null);

  const [form, setForm] = useState({
    materia_id: "",
    tipo_evaluacion: "",
    peso: "",
    descripcion: "",
    obligatoria: true,
  });

  useEffect(() => { cargarDatos(); }, []);

  async function cargarDatos() {
    setCargando(true);
    try {
      const [resConfigs, resMaterias] = await Promise.all([
        fetch("/api/admin/evaluacion"),
        fetch("/api/admin/materias"),
      ]);
      if (resConfigs.ok) {
        const data = await resConfigs.json();
        setConfigs(data.configuraciones);
      }
      if (resMaterias.ok) {
        setMaterias(await resMaterias.json());
      }
    } catch {
      toast.error("Error al cargar datos");
    } finally { setCargando(false); }
  }

  // Filtrar configs por materia seleccionada
  const configsFiltradas = materiaFiltro

```

```

    ? configs.filter((c) => c.materia_id === materiaFiltro)
    : configs;

// Calcular suma de la materia seleccionada
const sumaActual = materiaFiltro
  ? verificarSumaPesos(
      configs
        .filter((c) => c.materia_id === materiaFiltro)
        .map((c) => ({ peso: c.peso, tipo_evaluacion: c.tipo_evaluacion })))
  : null;

```

TypeScript - config-evaluacion-table.tsx (Parte 3: CRUD)

```

function abrirCrear() {
  setEditando(null);
  setForm({
    materia_id: materiaFiltro || "",
    tipo_evaluacion: "",
    peso: "",
    descripcion: "",
    obligatoria: true,
  });
  setDialogAbierto(true);
}

function abrirEditar(cfg: Config) {
  setEditando(cfg);
  setForm({
    materia_id: cfg.materia_id,
    tipo_evaluacion: cfg.tipo_evaluacion,
    peso: String(cfg.peso),
    descripcion: cfg.descripcion || "",
    obligatoria: cfg.obligatoria,
  });
  setDialogAbierto(true);
}

async function guardar() {
  const parsed = configEvaluacionSchema.safeParse({
    ...form,
    peso: parseFloat(form.peso),
  });
  if (!parsed.success) {
    toast.error(parsed.error.errors[0].message);
    return;
  }

  setGuardando(true);
  try {
    const esEdicion = editando !== null;

```

```

const url = esEdicion
  ? `/api/admin/evaluacion/${editando.id}`
  : "/api/admin/evaluacion";

const res = await fetch(url, {
  method: esEdicion ? "PUT" : "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(parsed.data),
});

if (!res.ok) {
  const err = await res.json();
  toast.error(err.error);
  return;
}

const data = await res.json();
toast.success(esEdicion ? "Actualizado" : "Creado");

// Mostrar advertencia si la suma no es 100
if (data.sumaPesos && !data.sumaPesos.esValida) {
  toast.warning(data.sumaPesos.mensaje);
}

setDialogAbierto(false);
cargarDatos();
} catch {
  toast.error("Error de conexión");
} finally { setGuardando(false); }
}

async function eliminar(id: string) {
  try {
    const res = await fetch(`/api/admin/evaluacion/${id}`, {
      method: "DELETE",
    });
    if (res.ok) {
      const data = await res.json();
      toast.success("Eliminado");
      if (data.sumaPesos && !data.sumaPesos.esValida) {
        toast.warning(data.sumaPesos.mensaje);
      }
      cargarDatos();
    }
  } catch {
    toast.error("Error al eliminar");
  }
}

// Helper: label del tipo

```

```
function labelTipo(tipo: string) {
  return TIPOS_EVALUACION.find((t) => t.value === tipo)?.label || tipo;
}
```

TypeScript - config-evaluacion-table.tsx (Parte 4: Render)

```
if (cargando) return <p className="text-muted-foreground p-6">Cargando...</p>;

return (
  <div className="space-y-4">
    {/* Selector de materia + indicador de suma */}
    <div className="flex items-center justify-between">
      <div className="flex items-center gap-4">
        <Select value={materiaFiltro} onChange={setMateriaFiltro}>
          <SelectTrigger className="w-[300px]">
            <SelectValue placeholder="Seleccionar materia" />
          </SelectTrigger>
          <SelectContent>
            {materias.map((m) => (
              <SelectItem key={m.id} value={m.id}>
                {m.codigo} - {m.nombre}
              </SelectItem>
            ))}
          </SelectContent>
        </Select>

        {/* Indicador de suma */}
        {sumaActual && (
          <div className="flex items-center gap-2">
            <Progress
              value={Math.min(sumaActual.suma, 100)}
              className="w-32 h-3"
            />
            <Badge className={
              sumaActual.esValida
                ? "bg-green-100 text-green-800"
                : sumaActual.suma > 100
                  ? "bg-red-100 text-red-800"
                  : "bg-yellow-100 text-yellow-800"
            }>
              {sumaActual.esValida
                ? <Check className="h-3 w-3 mr-1" />
                : <X className="h-3 w-3 mr-1" />
              }
              {sumaActual.suma}% / 100%
            </Badge>
          </div>
        )}
      </div>

      <Dialog open={dialogAbierto} onOpenChange={setDialogAbierto}>
```

```

<DialogTrigger asChild>
  <Button onClick={abrirCrear} disabled={!materiaFiltro}>
    <Plus className="h-4 w-4 mr-2" />
    Nueva Ponderación
  </Button>
</DialogTrigger>

  { /* DialogContent en Parte 5 */ }
</Dialog>
</div>

{!materiaFiltro && (
  <Card>
    <CardContent className="py-8 text-center text-muted-foreground">
      Selecciona una materia para ver sus ponderaciones.
    </CardContent>
  </Card>
)}

{materiaFiltro && (
  <Table>
    <TableHeader>
      <TableRow>
        <TableHead>Tipo</TableHead>
        <TableHead>Peso (%)</TableHead>
        <TableHead>Descripción</TableHead>
        <TableHead>Obligatoria</TableHead>
        <TableHead className="text-right">Acciones</TableHead>
      </TableRow>
    </TableHeader>
    <TableBody>
      {configsFiltradas.map((cfg) => (
        <TableRow key={cfg.id}>
          <TableCell className="font-medium">
            {labelTipo(cfg.tipo_evaluacion)}
          </TableCell>
          <TableCell>
            <Badge variant="outline" className="font-mono">
              {cfg.peso}%
            </Badge>
          </TableCell>
          <TableCell className="text-sm text-muted-foreground">
            {cfg.descripcion || "-"}
          </TableCell>
          <TableCell>
            {cfg.obligatoria ? (
              <Badge className="bg-blue-100 text-blue-800">Si</Badge>
            ) : (
              <Badge variant="outline">No</Badge>
            )}
          </TableCell>
          <TableCell className="text-right space-x-1">

```

```

        <Button variant="ghost" size="icon"
          onClick={() => abrirEditar(cfg)}>
          <Pencil className="h-4 w-4" />
        </Button>
        <AlertDialog>
          <AlertDialogTrigger asChild>
            <Button variant="ghost" size="icon">
              <Trash2 className="h-4 w-4 text-destructive" />
            </Button>
          </AlertDialogTrigger>
          <AlertDialogContent>
            <AlertDialogHeader>
              <AlertDialogTitle>
                Eliminar ponderación
              </AlertDialogTitle>
              <AlertDialogDescription>
                Se eliminará {labelTipo(cfg.tipo_evaluacion)}
                {" "}{(cfg.peso)%}. La suma de pesos cambiará.
              </AlertDialogDescription>
            </AlertDialogHeader>
            <AlertDialogFooter>
              <AlertDialogCancel>Cancelar</AlertDialogCancel>
              <AlertDialogAction
                className="bg-destructive text-destructive-foreground"
                onClick={() => eliminar(cfg.id)}
              >
                Eliminar
              </AlertDialogAction>
            </AlertDialogFooter>
          </AlertDialogContent>
        </AlertDialog>
      </TableCell>
    </TableRow>
  )})
  {configsFiltradas.length === 0 && (
    <TableRow>
      <TableCell colspan={5}
        className="text-center text-muted-foreground py-8">
        Sin ponderaciones configuradas para esta materia.
      </TableCell>
    </TableRow>
  )}
</TableBody>
</Table>
)}
</div>
);
}

```

TypeScript - config-evaluacion-table.tsx (Parte 5: DialogContent)

```

{/* Dentro del <Dialog> de Parte 4 */}

```

```

<DialogContent className="sm:max-w-[480px]">
  <DialogHeader>
    <DialogTitle>
      {editando ? "Editar Ponderacion" : "Nueva Ponderacion"}
    </DialogTitle>
    <DialogDescription>
      Define el peso de este tipo de evaluacion.
    </DialogDescription>
  </DialogHeader>

  <div className="grid gap-4 py-4">
    {/* Tipo de evaluacion */}
    <div className="grid gap-2">
      <Label>Tipo de Evaluacion</Label>
      <Select
        value={form.tipo_evaluacion}
        onChange={v =>
          setForm({ ...form, tipo_evaluacion: v })
        }
      >
        <SelectTrigger>
          <SelectValue placeholder="Seleccionar tipo" />
        </SelectTrigger>
        <SelectContent>
          {TIPOS_EVALUACION.map(t => (
            <SelectItem key={t.value} value={t.value}>
              {t.label}
            </SelectItem>
          ))}
        </SelectContent>
      </Select>
    </div>

    {/* Peso */}
    <div className="grid gap-2">
      <Label>Peso (%)</Label>
      <Input
        type="number"
        min="0"
        max="100"
        step="0.01"
        placeholder="Ej: 25.00"
        value={form.peso}
        onChange={e =>
          setForm({ ...form, peso: e.target.value })
        }
      />
    </div>

    {/* Descripcion */}
    <div className="grid gap-2">
      <Label>Descripcion (opcional)</Label>

```

```

        <Input
          placeholder="Ej: Primer examen parcial"
          value={form.descripcion}
          onChange={(e) =>
            setForm({ ...form, descripcion: e.target.value })
          }
        />
      </div>

      { /* Obligatoria */ }
      <div className="flex items-center gap-3">
        <Switch
          checked={form.obligatoria}
          onChange={(v) =>
            setForm({ ...form, obligatoria: v })
          }
        />
        <Label>Evaluación obligatoria</Label>
      </div>
    </div>

    <div className="flex justify-end">
      <Button onClick={guardar} disabled={guardando}>
        {guardando ? "Guardando..." : "Guardar"}
      </Button>
    </div>
  </DialogContent>

```

IMPORTANTE: Indicador visual de suma en tiempo real

El Badge junto al selector de materia muestra la suma actual con colores: verde (100% exacto), amarillo (< 100%), rojo (> 100%). Esto da feedback inmediato al admin sin necesidad de hacer clic en 'Guardar'. Se actualiza cada vez que se crea, edita o elimina una ponderación.

Paso 6: Componente Ponderaciones (Docente/Estudiante)

Componente reutilizable de solo lectura que muestra las ponderaciones de una materia. Lo usan tanto docentes como estudiantes.

TypeScript - components/dashboard/ponderaciones-materia.tsx (crear archivo nuevo)

```
"use client";

import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Badge } from "@components/ui/badge";
import { Progress } from "@components/ui/progress";
import { BarChart3, Check, X } from "lucide-react";
import { TIPOS_EVALUACION } from "@lib/validations/configuracion-evaluacion";

interface Config {
  tipo_evaluacion: string;
  peso: number;
  descripcion: string | null;
  obligatoria: boolean;
}

interface SumaPesos {
  suma: number;
  esValida: boolean;
  mensaje: string;
}

interface Props {
  materiaNombre: string;
  materiaCodigo: string;
  configuraciones: Config[];
  sumaPesos: SumaPesos;
}

export default function PonderacionesMateria({
  materiaNombre, materiaCodigo, configuraciones, sumaPesos,
}: Props) {
  function labelTipo(tipo: string) {
    return TIPOS_EVALUACION.find((t) => t.value === tipo)?.label || tipo;
  }

  return (
    <Card>
      <CardHeader>
        <div className="flex items-center justify-between">
          <CardTitle className="flex items-center gap-2 text-base">
            <BarChart3 className="h-4 w-4" />
```

```

    {materiaCodigo} - {materiaNombre}
  </CardTitle>
  <Badge className={
    sumaPesos.esValida
      ? "bg-green-100 text-green-800"
      : "bg-yellow-100 text-yellow-800"
  }>
    {sumaPesos.esValida
      ? <Check className="h-3 w-3 mr-1" />
      : <X className="h-3 w-3 mr-1" />
    }
    {sumaPesos.suma}%
  </Badge>
</div>
</CardHeader>
<CardContent>
  {configuraciones.length === 0 ? (
    <p className="text-muted-foreground text-sm">
      Aun no se han definido ponderaciones.
    </p>
  ) : (
    <div className="space-y-3">
      {configuraciones.map((cfg) => (
        <div key={cfg.tipo_evaluacion}
          className="flex items-center justify-between">
            <div className="flex-1">
              <div className="flex items-center gap-2">
                <span className="font-medium text-sm">
                  {labelTipo(cfg.tipo_evaluacion)}
                </span>
                {!cfg.obligatoria && (
                  <Badge variant="outline" className="text-xs">
                    Opcional
                  </Badge>
                )}
              </div>
              <div>
                {cfg.descripcion && (
                  <p className="text-xs text-muted-foreground mt-0.5">
                    {cfg.descripcion}
                  </p>
                )}
              </div>
            </div>
            <div className="flex items-center gap-2 ml-4">
              <Progress value={cfg.peso} className="w-20 h-2" />
              <span className="font-mono text-sm w-14 text-right">
                {cfg.peso}%
              </span>
            </div>
          </div>
        )})}
    </div>
  )}
</CardContent>

```

```
    </Card>  
  );  
}
```

Paso 7: Actualizar Páginas

TypeScript - app/(dashboard)/admin/evaluacion/page.tsx (reemplazar)

```
import ConfigEvaluacionTable from
  "@components/dashboard/admin/config-evaluacion-table";

export default function EvaluacionAdminPage() {
  return (
    <div className="p-6">
      <h1 className="text-2xl font-bold mb-6">
        Configuración de Evaluación
      </h1>
      <ConfigEvaluacionTable />
    </div>
  );
}
```

TypeScript - app/(dashboard)/docente/evaluacion/page.tsx (reemplazar)

```
"use client";

import { useEffect, useState } from "react";
import { toast } from "sonner";
import PonderacionesMateria from "@components/dashboard/ponderaciones-materia";

export default function EvaluacionDocentePage() {
  const [materias, setMaterias] = useState<any[]>([]);
  const [cargando, setCargando] = useState(true);

  useEffect(() => {
    async function cargar() {
      try {
        const res = await fetch("/api/docente/evaluacion");
        if (res.ok) {
          const data = await res.json();
          setMaterias(data.materias);
        }
      } catch {
        toast.error("Error al cargar ponderaciones");
      } finally {
        setCargando(false);
      }
    }
    cargar();
  }, []);

  if (cargando) return <p className="p-6 text-muted-foreground">Cargando...</p>;

  return (
    <div className="p-6">
```

```

<h1 className="text-2xl font-bold mb-6">Ponderaciones de Mis Materias</h1>
<div className="space-y-4">
  {materias.map((m) => (
    <PonderacionesMateria
      key={m.materia.id}
      materiaNombre={m.materia.nombre}
      materiaCodigo={m.materia.codigo}
      configuraciones={m.configuraciones}
      sumaPesos={m.sumaPesos}
    />
  ))}
  {materias.length === 0 && (
    <p className="text-muted-foreground">No tienes materias asignadas.</p>
  )}
</div>
</div>
);
}

```

NOTA: Switch de shadcn/ui

El formulario usa el componente Switch para el campo 'obligatoria'. Si no lo tienes instalado, ejecuta:
 npx shadcn@latest add switch

Mapa Completo de Archivos

Archivo	Tipo	Descripción
lib/validations/configuracion-evaluacion.ts	Schema	Zod schema + verificarSumaPesos() + constantes
app/api/admin/evaluacion/route.ts	API Route	GET (listar con suma) + POST (crear con suma)
app/api/admin/evaluacion/[id]/route.ts	API Route	PUT (editar con suma) + DELETE (eliminar con suma)
app/api/docente/evaluacion/route.ts	API Route	GET agrupado por materia del docente
app/api/estudiante/evaluacion/route.ts	API Route	GET agrupado por materias inscritas
components/.../admin/config-evaluacion-table.tsx	Componente	Tabla CRUD con indicador de suma en tiempo real
components/.../ponderaciones-materia.tsx	Componente	Card de solo lectura con barras de peso
app/(dashboard)/admin/evaluacion/page.tsx	Página	Actualizada con ConfigEvaluacionTable
app/(dashboard)/docente/evaluacion/page.tsx	Página	Actualizada con PonderacionesMateria

Verificación Final

- ✓ lib/validations/configuracion-evaluacion.ts exporta schema, TIPOS_EVALUACION y verificarSumaPesos.
- ✓ GET /api/admin/evaluacion devuelve configuraciones[] y resumenPorMateria con sumas.
- ✓ POST /api/admin/evaluacion crea registro y devuelve sumaPesos actualizada.
- ✓ PUT y DELETE en /api/admin/evaluacion/[id] actualizan y devuelven sumaPesos.
- ✓ GET /api/docente/evaluacion devuelve materias[] agrupadas con configuraciones y sumaPesos.
- ✓ GET /api/estudiante/evaluacion devuelve materias inscritas con ponderaciones.
- ✓ El indicador de suma cambia de color en tiempo real: verde (100%), amarillo (<100%), rojo (>100%).
- ✓ El toast.warning se muestra al guardar si la suma no es exactamente 100%.
- ✓ El Dialog con Switch para 'obligatoria' funciona correctamente.
- ✓ Error 409 se maneja al intentar agregar un tipo de evaluación duplicado.
- ✓ PonderacionesMateria muestra barras de progreso por tipo en vista docente.
- ✓ npm run dev funciona sin errores. Switch instalado (npx shadcn@latest add switch).

Ejercicio Propuesto

1. Tarea 1: Agrega un botón 'Plantilla Rápida' que pre-cargue una configuración estándar (parcial_1: 25%, parcial_2: 25%, practica: 25%, proyecto: 25%) con un solo click. Pista: haz 4 POST secuenciales o modifica la API para aceptar un array.
2. Tarea 2: Modifica la vista del estudiante para que también muestre las ponderaciones de evaluación debajo de cada materia en la página de Mis Asistencias (combinando datos de ambos endpoints).
3. Tarea 3: Agrega validación visual al Dialog: si el peso ingresado haría que la suma exceda 100%, muestra un warning inline ANTES de guardar. Pista: calcula la suma proyectada en el onChange del input peso.
4. Tarea 4: Implementa un endpoint GET /api/admin/evaluacion/incompletas que devuelva las materias cuya suma de pesos NO es 100%. Muestra un badge de alerta en el sidebar junto a 'Evaluación'.

Proximo: Entrega 10B

En la Entrega 10B cerraremos la Fase 4 con: integración de las nuevas métricas en los dashboards existentes, cálculo de nota final ponderada usando configuración_evaluacion + calificaciones, checklist final de la Fase 4, y preparación del resumen de entregas 8A-10B para documentación del proyecto.

Que vamos a construir	Archivos
Cálculo de nota final ponderada	lib/utils/calcular-nota-final.ts
API endpoint nota final por estudiante	app/api/docente/notas-finales/route.ts
Componente vista de notas finales	components/.../notas-finales-table.tsx
Actualización de dashboards con métricas Fase 4	Actualizar páginas admin/docente/estudiante
Checklist final + resumen Fase 4	Documento de cierre